CIBERSEGURIDAD / PHP RUNTIME / INFORME TECNICO

# PHP2Shell: cuando falla la memoria del motor PHP

Dos vulnerabilidades críticas, dos contenedores y una guía paso a paso para comprender el riesgo de ejecución remota de código.

CVE-2026-109618 OOM y estado no inicializado | CVE-2026-109619 Referencias obsoletas entre peticiones
Gravedad oficial: 9,2 CVSS v4.0 | Divulgación: 21 de octubre de 2026

## Resumen ejecutivo

Ambos fallos afectan al núcleo de PHP, no a una función insegura introducida por una aplicación concreta. El primero combina errores de asignación de memoria con un abandono del flujo mediante longjmp(). El segundo puede dejar referencias globales apuntando a memoria ya liberada, que se usa en otra petición del mismo worker.

Los avisos del proyecto PHP confirman potencial de RCE sin autenticación, especialmente en Apache mod_php. La ejecución fiable depende de condiciones técnicas y manipulación especializada de memoria; una versión vulnerable o un PID repetido no constituyen por sí mismos una explotación.

### 01 / CVE-2026-109618

Inicialización parcial + agotamiento de memoria + limpieza en estado inconsistente.

### 02 / CVE-2026-109619

Estado global retenido + memoria de petición liberada + uso posterior (UAF).

Qué incluye el material descargable. Dos contenedores PHP 8.3.35-apache, comandos para observar parámetros y workers, y una emulación de impacto claramente etiquetada. No contiene un exploit funcional de las CVE.

Autor: Alejandro Aránguez Romera · 22.10.2026 · Material público de divulgación y aprendizaje.

# Qué falla dentro de PHP

## CVE-2026-109618 — Out of Memory y longjmp()

Si una asignación de memoria falla durante la preparación de una petición, el salto de error puede dejar una estructura de solicitud incompletamente inicializada. El cierre de la petición en Apache mod_php puede ejecutar operaciones de limpieza sobre ese estado inválido incluso antes de llegar al script PHP.

> HTTP multipart → Asignación falla → longjmp()
> Estado incompleto → Limpieza → Corrupción posible → RCE potencial

## CVE-2026-109619 — Estado global y use-after-free

Un worker guarda temporalmente estado de errores, sesiones u otros subsistemas. Si una ruta de error evita restablecer una referencia, la memoria de la petición puede liberarse dejando un puntero global colgante. Una operación temprana en otra petición atendida por el mismo proceso puede usar ese estado.

> Petición A (productor) → Estado global apunta al heap A
> Fin de A / memoria liberada → Petición B (consumidor) → UAF posible

## Parámetros que conviene inspeccionar

| Elemento | Relación con la superficie vulnerable |
|---|---|
| memory_limit | Límite de PHP relevante para condiciones OOM. |
| post_max_size | Control del tamaño de POST; no mide por sí solo todas las estructuras internas. |
| max_file_uploads / max_input_nesting_level | Límites para cargas y profundidad de variables de entrada. |
| session.upload_progress.enabled | Función implicada en rutas confirmadas de estado global obsoleto. |
| Apache mod_php / worker persistente | Procesamiento y limpieza de memoria, reutilización entre peticiones. |

Los valores del laboratorio son intencionadamente sencillos: memory_limit=128M, post_max_size=8M, upload_max_filesize=2M. Los límites reducen exposición en distintos escenarios, pero no reemplazan la actualización.

# Dos Docker, cuatro pruebas

Los servicios se construyen con la imagen oficial php:8.3.35-apache y se publican exclusivamente en 127.0.0.1. El ejercicio inspecciona el entorno sin provocar agotamiento de memoria, bloqueos ni corrupción.

| Contenedor / Puerto | Qué se observa |
|---|---|
| CVE-2026-109618 / 18118 | Directivas de PHP, envío multipart normal y campos recibidos. |
| CVE-2026-109619 / 18119 | Mismo PID de worker entre peticiones; impacto emulado aparte. |

## 1. Arrancar ambos contenedores

```
unzip PHP2Shell_LinkedIn_Lab.zip
cd PHP2Shell_LinkedIn_Lab
./00_iniciar.sh
```

## 2. Inspeccionar el parser multipart

```
curl -sS -F 'tema=PHP2Shell' \
  http://127.0.0.1:18118/inspeccionar.php \
  | python3 -m json.tool
```

Observa peticion.campos_post_recibidos, directivas.memory_limit, post_max_size y sapi=apache2handler. El registro de memoria es posterior al parseo, por lo que no demuestra OOM.

## 3. Observar reutilización de un worker

```
for i in 1 2 3; do
  curl -sS http://127.0.0.1:18119/inspeccionar.php \
  | python3 -c 'import json,sys;
      print(json.load(sys.stdin)["worker_pid"])'
done
```

Con un solo worker hijo, el PID se mantiene. Esto representa la persistencia del proceso necesaria para ciertas rutas de UAF, pero no prueba la existencia de una referencia colgante.

## 4. Emulación del impacto de una RCE

```
curl -sS 'http://127.0.0.1:18119/impacto_controlado.php?accion=identidad'
curl -sS 'http://127.0.0.1:18119/impacto_controlado.php?accion=crear_marca'
docker compose exec -T cve-109619 cat /tmp/php2shell_marca_educativa.txt
```

> Importante: el endpoint ejecuta únicamente acciones predefinidas mediante PHP. La ejecución y escritura son reales dentro de Docker, pero NO son el resultado de explotar ninguna de las dos CVE.

# De la corrupción de memoria al riesgo operativo

## ¿Por qué puede aparecer RCE?

PHP ejecuta componentes nativos que utilizan punteros, estructuras de heap y ciclos de asignación y liberación. En determinados contextos, el uso de memoria no inicializada o liberada puede pasar de provocar un bloqueo a permitir desviar el flujo nativo de ejecución. El proyecto PHP documenta RCE sin autenticación contra Apache mod_php. Ello requiere un camino de explotación específico y no se deduce de un formulario normal.

| Impacto plausible con RCE real | Límite de interpretación |
|---|---|
| Identidad de ejecución del worker web | No implica control como root ni privilegios del host. |
| Lectura de ficheros accesibles al worker | Depende de permisos, montaje y aislamiento. |
| Escritura en rutas habilitadas | No significa que todos los directorios sean escribibles. |
| Disponibilidad: caída/reinicio de procesos | No toda corrupción alcanza ejecución de comandos. |

## Remediaciones verificadas por el proveedor

> Actualizar PHP: 8.2.35 · 8.3.36 · 8.4.27 · 8.5.12 (o posteriores soportadas).
> Reiniciar workers, verificar la versión y el SAPI realmente desplegados, revisar procesamiento multipart, sesión, diagnósticos tempranos y políticas de entrada. Ninguna configuración aislada sustituye al parche.

## Alcance experimental y honestidad de la PoC

Los Docker emplean una versión afectada, muestran rutas de entrada y persistencia del worker, e incluyen una demo delimitada de acciones posibles si existiera ejecución de código. No producen una falla OOM maliciosa, UAF, corrupción de memoria ni RCE real mediante las CVE. Por ello, el laboratorio no permite certificar la explotabilidad de una infraestructura específica.

## Fuentes primarias (publicadas el 21/10/2026)

1. PHP — CVE-2026-109618: github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w
2. PHP — CVE-2026-109619: github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp
3. PHP — Versiones soportadas: php.net/supported-versions.php

---

Mensaje final: una aplicación puede no tener funciones peligrosas y, aun así, ejecutarse sobre un runtime expuesto. Inventariar versiones y SAPI, corregir el motor y entender sus fases de error es una prioridad de seguridad.